

BWC Adaptive Selector: Pair-Aware Runtime Prefetcher Selection in ChampSim

Zhaofeng Luo

Kijun Shin

15-740 Computer Architecture

0. Abstract

Hardware prefetchers hide memory latency, but aggressive prefetch traffic can consume shared cache capacity, memory queues, and off-chip bandwidth. This is especially important in multicore systems, where a locally plausible prefetch stream can delay demand requests from a peer. We address this prefetch-control problem in ChampSim by building the BWC Adaptive Selector, a pair-aware L2 runtime selector over original SPP, an FDP-style SPP variant, and BOP. BWC uses local stream features and peer/shared signals to decide whether the local expert choice is safe under contention. Across Phase 1 scale sweeps at 1M+5M, 2M+10M, and 4M+20M, average gains remain positive for both single-core IPC (+1.5464%, +1.8540%, +2.6782%) and two-core weighted speedup (+1.3677%, +1.6013%, +1.3017%). Individual rows are mixed, especially for some `lbm`-related pairs and near-neutral `bzip2` and `lbm` single-core cases. OpenEvolve was then used in a focused Phase 2 2M+10M diagnostic to refine only pair coordination, improving `bzip2` + `lbm` and reducing the `mcf` + `lbm` worst-case loss. The main insight is that prefetch control should be peer-aware: locally reasonable prefetch decisions can become globally harmful under shared-resource contention.

1. Introduction

Modern data prefetchers exploit regular memory behavior to fetch lines before the core demands them. This can substantially improve performance for streaming and structured workloads. However, prefetches are not free: each request occupies queues, MSHRs, cache space, and DRAM bandwidth. In multicore systems, these costs cross core boundaries. A prefetcher that helps one core locally can hurt another core through shared LLC and memory-system pressure.

The project began from a bandwidth-aware control question: can a runtime policy control an SPP-like prefetcher better than static or accuracy-only throttling? Accuracy is useful but incomplete. A low-accuracy stream can still provide coverage or memory-level effects, while a confident predictor can still create harmful traffic. We therefore treat prefetching as a control problem, not only an address-prediction problem.

The final artifact is the BWC Adaptive Selector. Rather than inventing a new predictor, it chooses among experts with different behaviors: `spp_orig`, `spp_dev` with FDP-style controls, and `bop`. The selector asks two questions on every stable window: what does the local stream look like, and does the peer make the locally preferred choice risky? Our research question is whether these local and peer/shared signals can choose safer prefetch behavior than a single static policy. The scale sweep shows positive average behavior over a static `SPP_orig` baseline at all tested simulation lengths, while also exposing fragility in individual cases. In particular, longer runs reveal small regressions for several `lbm`-related pairs, so the final conclusion is not that every case is solved; it is that peer-aware selection improves the average tradeoff and points clearly to the remaining hard interactions.

2. Related Work

Feedback Directed Prefetching (FDP) by Srinath et al. dynamically adjusts prefetch aggressiveness using feedback such as accuracy, timeliness, and pollution [1]. Our `spp_dev` expert contains an FDP-style controller, but the final design builds above it: it selects among experts and uses pair signals to avoid choices that are unsafe under shared contention.

The Signature Path Prefetcher (SPP), based on path-confidence lookahead prefetching, is our main SPP baseline [2]. It uses signatures and confidence to look ahead in access streams. Best-Offset Prefetching (BOP) searches for spatial offsets that work for the current stream [3]. In our design, these mechanisms are not only baselines but also expert behaviors: SPP represents the strong default lookahead prefetcher, the FDP-style variant represents a feedback-controlled SPP mode, and BOP represents an offset-oriented alternative for streams where it behaves differently. BWC differs by adding a selector and pair coordinator above the experts rather than replacing their address-generation logic. ChampSim is the trace-based simulation framework used to evaluate these designs [4]. OpenEvolve is an evolutionary coding framework with LLM-guided mutation and quality-diversity search [5]. We use it in a narrow, coordinator-only way so the evolved design remains interpretable.

3. Method

Implementation Overview

The main implementation is in `prefetcher/adaptive_selector/`. The Phase 2 retained best point is stored separately in `prefetcher/adaptive_selector_phase2_best/`. Table 1 maps the main selector stages to source-level artifacts.

Table 1: Source artifacts and implementation map.

Stage	Main function / artifact	Role
Feature extraction	<code>compute_window_features()</code>	Computes <code>page_growth</code> , <code>line_growth</code> , <code>small_delta_ratio</code> , and <code>rfo_share</code> .
Local selection	<code>classify_window()</code>	Produces the local <code>orig</code> , FDP, or BOP candidate.
Shared context	<code>refresh_shared_pressure()</code>	Records local pressure plus peer page/delta signatures and <code>peer_1bm_like</code> .
Pair coordination	<code>coordinate_candidate()</code>	Remaps or blocks risky pair choices before activation.
Configurations	<code>adaptive_selector</code> configs	Instantiate the selector in single-core and two-core ChampSim runs.

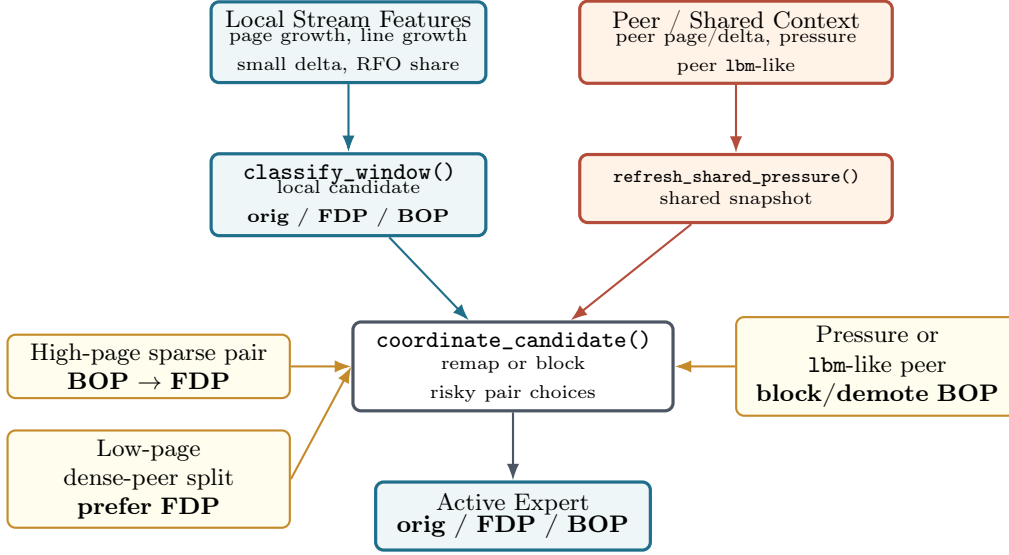
**Figure 1:** Simplified BWC Adaptive Selector pipeline, based on the poster flow. Local stream features select a candidate; peer/shared signals guard the final activation.

Figure 1 shows the final design. Each demand access is pushed into a sliding window, and the window produces four local features. A local classifier proposes an expert. In two-core mode, shared snapshots record peer behavior and pressure. The coordinator then applies pair-aware rules before the candidate becomes the active expert. We use the name FDP for the FDP-style SPP expert in the selector; it is not a full reimplement of the FDP paper, but an SPP variant with feedback-style controls intended to be safer under some traffic patterns. Overall, the simulated policy is relatively hardware-conscious: it relies on short counters, ratios, threshold comparisons, and a small shared state vector rather than expensive global history.

OpenEvolve Setup

The original proposal planned a parameter-only OpenEvolve search over an SPP controller. During development, the retained artifact shifted to expert selection because it gave clearer control over heterogeneous pair behavior. For Phase 2, OpenEvolve searched only the coordinator logic. The expert prefetchers stayed fixed. The best retained point changed several pair fallbacks from `orig` to FDP, including dense-peer split handling, pair-scope BOP demotion, and `1bm`-like peer protection. It also added two runtime guards for sparse/high-page-growth cores paired with `1bm`-like peers, and for `1bm`-like cores paired with high-page-growth peers.

For OpenEvolve, we constrained mutations to the pair-coordination decision path, primarily `coordinate_candidate()`, and kept the expert prefetchers and ChampSim interfaces fixed. The prompt emphasized preserving the Phase 1 selector behavior while improving fragile `1bm`-related pair cases. Each candidate was evaluated on a focused 2M-warmup plus 10M-simulation mini-suite containing single-core `1bm`, `bzip2 + 1bm`, and `mcf + 1bm`. The score combined mean gain with a worst-case guardrail so that a candidate had to improve pair behavior without introducing a new large regression. We retained the best candidate according to this focused score and report the broader final performance summary separately. The reported combined score is this internal OpenEvolve ranking objective; higher is better, and it is used only to rank Phase 2 candidates rather than as a separate architectural metric.

Evaluation Setup

Table 2: Evaluation setup.

Item	Setting
Simulator	ChampSim trace-based simulation.
Primary baseline	SPP_orig; reported gains are relative to this baseline.
Selector configs	configs/adaptive_selector_config.json and configs/adaptive_selector_2core.json.
Experts	Original SPP, FDP-style SPP, and BOP.
Workloads	astar, mcf, lbm, and bzip2.
Metrics	Single-core IPC and two-core weighted speedup.
Weighted-speedup definition	Sum of each core’s shared-run IPC normalized to its corresponding single-core IPC; gains are relative to the SPP_orig pair baseline.
Phase 1 scale sweep	1M+5M, 2M+10M, and 4M+20M; baseline is spp_orig, current is phase 1 final adaptive_selector.
Sweep execution	Completed on office-kijun with at most 3 sweep jobs in parallel.
Phase 2 run length	2M warmup plus 10M simulation for the focused OpenEvolve comparison.

4. Experimental Results

Phase 1 scale sweep. The main final result is a scale sweep of the Phase 1 final selector against spp_orig. Table 3 and Figure 2 show that average single-core and two-core gains stay positive at all three tested lengths. The averages hide important variation, so Tables 4 and 5 report the per-workload and per-pair gain matrices.

Table 3: Phase 1 final scale sweep average gains.

Scale	Single-core average IPC gain	Two-core average WS gain
1M+5M	+1.5464%	+1.3677%
2M+10M	+1.8540%	+1.6013%
4M+20M	+2.6782%	+1.3017%

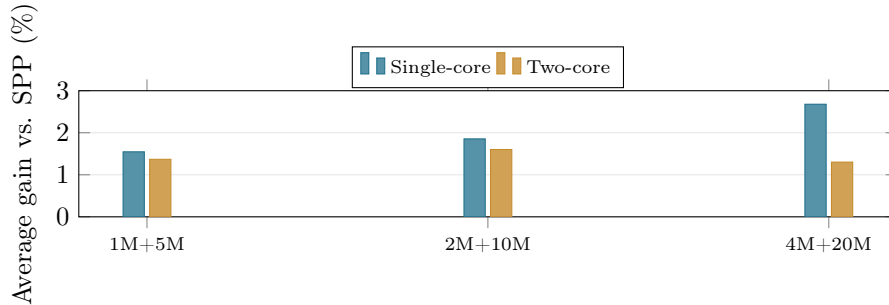


Figure 2: Scale-sweep average gains. Average behavior remains positive, but the detailed matrices show that several individual cases are still fragile.

Table 4: Single-core IPC gain matrix for the Phase 1 final selector.

Workload	1M+5M	2M+10M	4M+20M
astar	+2.6292%	+3.8644%	+5.9008%
mcf	+3.7178%	+3.1798%	+4.8350%
lbm	-0.0708%	+0.3722%	-0.0165%
bzip2	-0.0907%	-0.0006%	-0.0066%

The single-core sweep is strongest on **astar** and **mcf**; **lbm** and **bzip2** are near-neutral and sometimes slightly negative. This suggests the selector’s local feature rules help irregular/memory-sensitive workloads more reliably than already-stable dense or compute-heavy streams.

Table 5: Two-core weighted-speedup gain matrix. Negative entries are highlighted.

Pair	1M+5M	2M+10M	4M+20M
mcf + astar	+2.6541%	+4.3939%	+4.6815%
astar + bzip2	+1.5328%	+2.2457%	+2.7770%
bzip2 + mcf	+1.3779%	+2.1973%	+1.2700%
astar + lbm	+1.4340%	+1.3858%	-0.1283%
bzip2 + lbm	+0.5820%	-0.0286%	-0.1448%
mcf + lbm	-0.1211%	-1.0157%	-0.6557%
astar + astar	+4.2142%	+4.9374%	+4.1861%
mcf + mcf	+1.7809%	+1.8082%	+1.1320%
lbm + lbm	+0.1705%	+0.0706%	-0.1168%
bzip2 + bzip2	+0.0515%	+0.0187%	+0.0155%

The two-core averages remain positive, but longer runs expose the hard cases: **mcf + lbm** is negative at every scale, and several other **lbm** pairs turn negative at 4M+20M. This is the clearest evidence that peer-aware control is useful but incomplete.

Phase 2 OpenEvolve diagnostic. Phase 2 is a focused 2M+10M diagnostic rather than a replacement for the scale sweep. It targets the fragile 1bm cases exposed by the sweep. The best candidate changes only `coordinate_candidate()` and improves the two target pair summaries, as shown in Table 6 and Figure 3.

Table 6: Phase 2 OpenEvolve best point.

Case	SPP base	Phase 1	P1 gain	Phase 2	P2 gain	Delta
1bm IPC	0.866441	0.869666	+0.3722%	0.869666	+0.3722%	+0.0000 pp
bzip2 + 1bm WS	1.792235	1.791723	-0.0286%	1.793705	+0.0820%	+0.1106 pp
mcf + 1bm WS	1.256300	1.243540	-1.0157%	1.249503	-0.5410%	+0.4747 pp

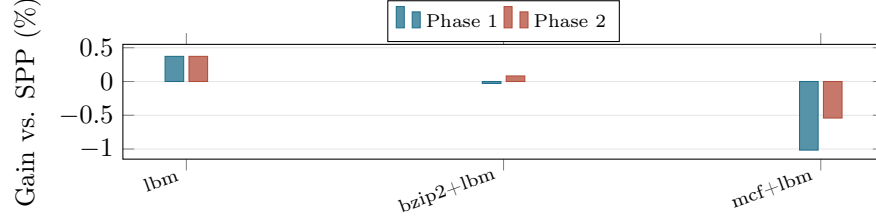


Figure 3: Phase 2 before/after gains. OpenEvolve fixes the small `bzip2 + 1bm` regression and reduces the `mcf + 1bm` loss from -1.0157% to -0.5410%.

The aggregate internal ranking score improves from -6.2665 to -3.0105, where higher is better. Mean gain improves by 0.1951 percentage points and worst-case gain by 0.4747 percentage points.

Experimental process. The process was iterative: local selection produced useful wins, pair-aware coordination protected several mixed workloads, the scale sweep exposed remaining 1bm fragility, and Phase 2 used coordinator-only OpenEvolve mutations to reduce those failures.

Table 7: Experimental progression leading to the final report state.

Stage	What it established
Sliding-window selector	Local stream features could choose among <code>orig</code> , <code>FDP</code> , and <code>BOP</code> without changing expert internals.
Pair-aware coordination	Some locally plausible choices were harmful under shared pressure, especially when an 1bm-like peer was present.
Scale sweep	Average gains remained positive at 1M+5M, 2M+10M, and 4M+20M, but individual 1bm pairs exposed stability limits.
OpenEvolve Phase 2	Coordinator-only mutations improved the focused <code>bzip2 + 1bm</code> and <code>mcf + 1bm</code> diagnostics without changing expert predictors.

5. Goals and Next Steps

The original proposal goal was an OpenEvolve-optimized bandwidth-aware SPP controller. We partially met that goal, but the final artifact shifted to a stronger adaptive-selector formulation. Instead of tuning a single SPP controller, we select among SPP, FDP-style SPP, and BOP. This better matches the observed problem: different workloads and pairs need different expert styles, not only different throttling thresholds. Although this shift changed the mechanism, it preserved the proposal’s larger goal: using automated search to improve prefetch control under shared-resource pressure.

Future work should evaluate more held-out SPEC mixes, run longer and repeated simulations, validate four-core behavior, and use OpenEvolve over a broader but still constrained coordinator rule space. The scale sweep makes the most useful next analysis clear: ablate each pair rule and report prefetch traffic, cache pollution, and queue pressure alongside IPC and weighted speedup.

6. Collaboration

Zhaofeng Luo focused on experiment infrastructure, running ChampSim evaluations, collecting IPC and weighted-speedup summaries, and preparing the final result tables and poster figures. He also led the final report and poster writing, helped analyze the single-core and two-core results, and validated the retained result ledger.

Kijun Shin focused on the adaptive-selector implementation, pair-coordination rules, and OpenEvolve-guided Phase 2 refinement. He also helped debug the coordinator behavior and interpret the evolved changes to `coordinate_candidate()`. Both members jointly designed the BWC selector idea, studied related prefetching work, discussed evaluation methodology, reviewed intermediate results, and refined the final project narrative.

7. Conclusion

The BWC Adaptive Selector demonstrates that prefetch control should not be purely local. The selector uses local stream features to choose an expert and peer/shared signals to decide whether that choice is safe for the memory system. The scale sweep shows positive average gains at all tested lengths, but also exposes remaining fragility in individual 1bm-related cases. The Phase 2 OpenEvolve best point further shows that narrow, coordinator-only evolution can reduce those difficult interactions without changing expert predictor internals. The central lesson is simple: the right prefetcher for one core is not always the right prefetcher for the system.

References

- [1] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt. Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers. HPCA, 2007. https://hps.ece.utexas.edu/pub/srinath_hPCA07.pdf

- [2] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti. Path Confidence Based Lookahead Prefetching. MICRO, 2016. <https://dblp.org/rec/conf/micro/KimPGRWC16.html>
- [3] P. Michaud. Best-Offset Hardware Prefetching. HPCA, 2016. <https://core.ac.uk/outputs/48160133/>
- [4] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. H. Pugsley, and J. Kim. The Championship Simulator: Architectural Simulation for Education and Competition. arXiv:2210.14324, 2022. <https://arxiv.org/abs/2210.14324>
- [5] OpenEvolve project repository. <https://github.com/algorithmicsuperintelligence/openevolve>. Accessed Apr. 2026.